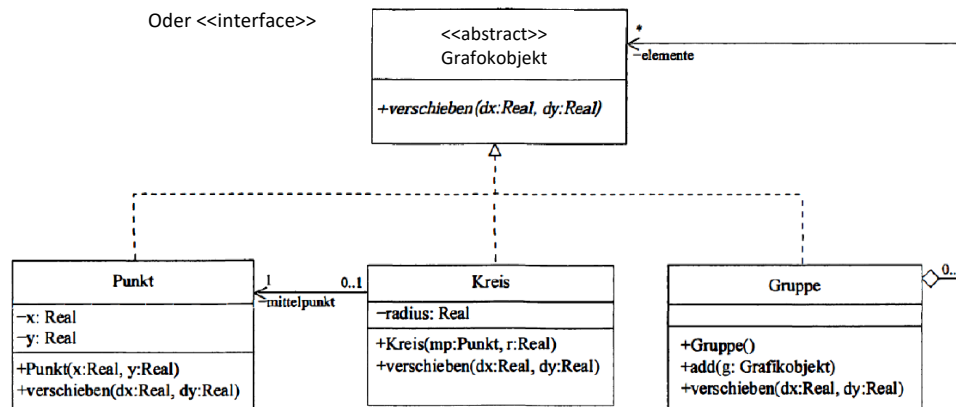


Das Entwurfsmuster Kompositum – Composite Pattern

Das Composite-Pattern wird in erster Linie dazu verwendet, um Fallunterscheidungen zu vermeiden. Dazu werden alle Gemeinsamkeiten der Klassen in einem Interface bzw. einer abstrakten Klasse zusammengefasst. Zusätzlich existiert dann für jede mögliche Ausprägung eine eigene Klasse mit eigenem Verhalten für die in der abstrakten Klasse / Interface festgelegten Methoden.



Eine **abstrakte Klasse** unterscheidet sich von einer regulären Klasse nur insoweit, als dass sie nicht _____ werden kann – man kann also kein Objekt dieser Klasse bauen. Dennoch besitzen abstrakte Klasse einen _____, da ihre Subklassen den Super-Konstruktor Aufruf `super()` benötigen um initialisiert werden zu können. **Methoden und Attribute können** weiterhin wie gewohnt umgesetzt werden (oder explizit **abstract** deklariert werden -> Stub). In Klassen, die von der abstrakten Klasse erben, kann dann die Standardimplementierung aus der Oberklasse genutzt werden oder die Methode mittels **@Override** überschrieben werden, um die Funktionsweise zu verändern. Dies ist von **Interfaces** abzugrenzen, da diese nur „Stubs“ – hier **Methodensignaturen** – enthalten können, keine Implementationen. Diese müssen dann in den Unterklassen implementiert werden; Das Überschreiben der Methoden aus der abstrakten Oberklasse ist hingegen optional. Interfaces besitzen außerdem keinen Konstruktor.

Liste in perfekter Komposition

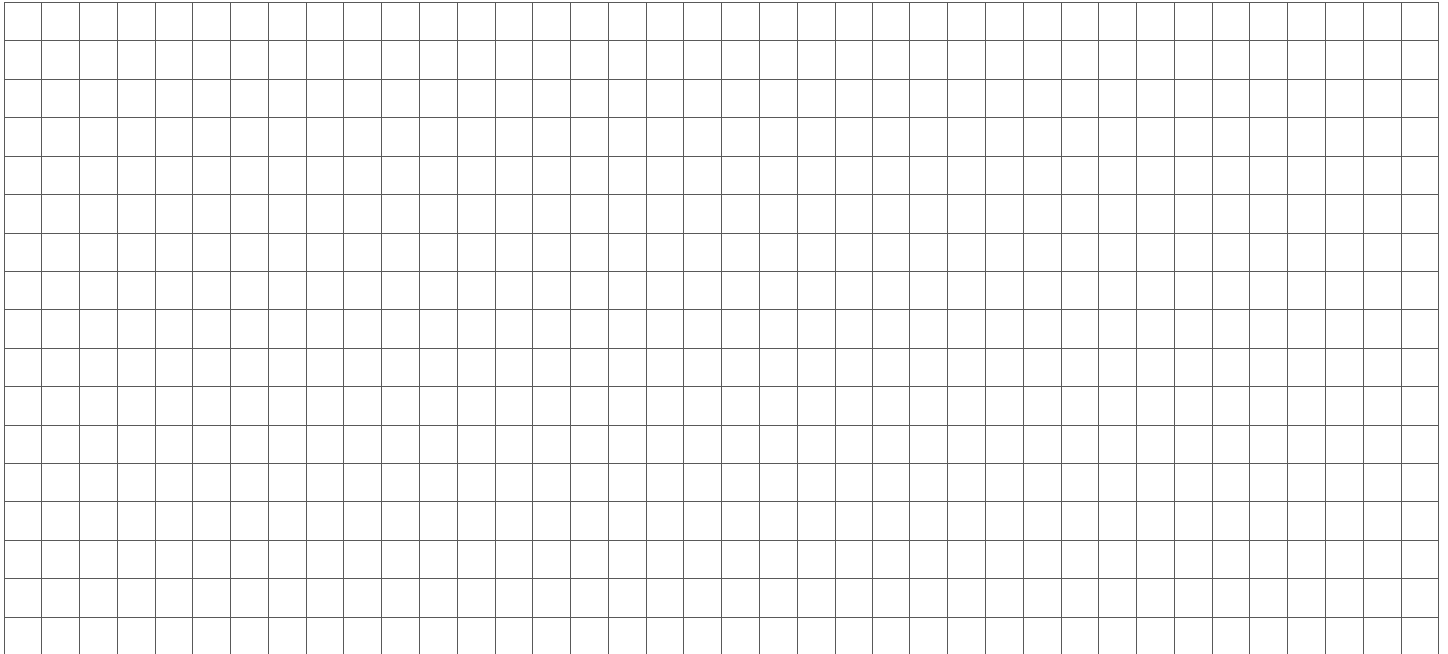
Im Fall der Liste möchten wir die Fallunterscheidungen bei der Verwaltung der Rekursionsfälle für das letzte Listenelement vermeiden. Statt also den **nachfolger** auf `null` zu prüfen, wird nun die Logik aufgeteilt in zwei verschiedene Klassen: **Knoten** und **Abschluss**, die von der abstrakten Klasse **ListeElement** erben.

Aufgabe 1: Erklären Sie, an welchen Stellen im Code nun Datentypen geändert werden müssen (von – zu) und warum.

Leere Liste: In diesem Fall referenziert das Attribut **anfang** einfach den Abschlussknoten.

Aufgabe 2: Zeichnen Sie das Objektdiagramm einer Liste mit 2 Knoten und einer leeren Liste.

Aufgabe 3: Zeichnen Sie ein vereinfachtes Klassendiagramm mit der neuen Klasse **Abschluss** und der abstrakten Klasse **ListenElement** sowie den bestehenden Klassen für **Liste**, **Knoten**, **DatenElement** und **Patient/Taxi/Kunde/...** (vgl. S. 36)



Die Methodensignatur in **ListenElement** – beachte geänderten Rückgabetyt!

```
abstract ListenElement add(DatenElement daten);
```

Add Methode in der **Knoten** Klasse

```
@Override
ListenElement add(DatenElement daten) {
    nachfolger = nachfolger.add(daten);
    return this;
}
```

Verkettung von Knoten muss passen

Reihenfolge herstellen; falls neuer Nachfolger

Reihenfolge (Richtung Anfang) beibehalten

Add Methode in der **Abschluss** Klasse

```
@Override
ListenElement add(DatenElement daten)
{
    Knoten temp = new Knoten(daten);
    temp.setNachfolger(this);
    return temp;
}
```

Am Ende angekommen

Neuen Knoten mit entsprechenden Daten erzeugen.

Setze den Abschluss als Nachfolger des neuen Knotens

Gib temp als neuen Nachfolger für "Vorgänger" zurück

Aufgaben 4a) Erkunden Sie das Projekt 12-1-4-Liste mit Kompositum und markieren Sie sich weitere zu ändernde Methoden mit `//TODO`.

b) Vergleichen Sie hierzu auch im Buch Seiten 32-41. Notieren Sie weitere Methoden.

c) Vergleichen Sie die offizielle Oracle JavaDoc zur `LinkedList`.
